Kaiku — Technical Documentation

Elisa × Since AI 2026 Challenge · *Capturing and Linking Human-Generated Contextual Metadata for Autonomous Network Operations* · Nov 8, 2026

Executive summary

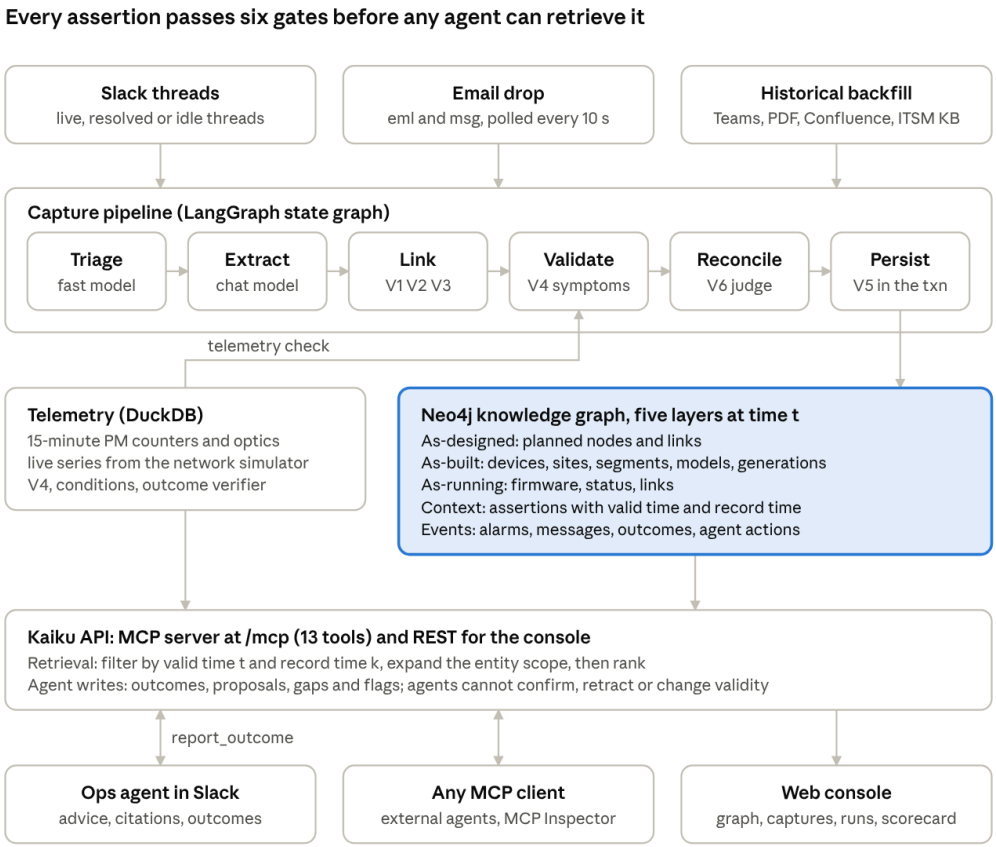
Kaiku turns operational knowledge that experts share in chat and email into time-bound, network-verified context that any AI agent can query over the Model Context Protocol (MCP). It was built for Elisa's Since AI 2026 challenge, *Capturing and Linking Human-Generated Contextual Metadata for Autonomous Network Operations*.

Problem. Critical operating rules live only in conversations. Elisa's reference case: after an SFP replacement on TRN-042 the port is UP, optics are normal and alarms have cleared, yet traffic stays at zero. The expert knows the fix is an interface disable/enable, not another SFP swap or a reboot, but that rule exists only in a Teams chat and an email. A second case: a field engineer dispatched to a Northern Finland site finds a locked barrier whose key location was only ever emailed.

Approach. Kaiku watches Slack channels, an email drop folder and historical sources (Teams export, Outlook `.msg`, PDF notes, Confluence, ITSM KB). A small model triages each resolved or idle thread; a chat model extracts structured *assertions* (conditions, action, avoid list, scope, validity). Six deterministic gates (V1–V6) verify every assertion against its source text, the network inventory and live telemetry before it is written to a Neo4j graph with explicit valid-time and record-time. Agents retrieve it through 13 MCP tools, and reported outcomes feed back into a Bayesian confidence score.

System architecture

Kaiku is a pipeline from conversations to a bitemporal knowledge graph, with MCP as the single read and write surface for agents. It runs as six Docker Compose services: Neo4j, the API (REST and MCP), the Slack bot, the network simulator, the web console and Phoenix for tracing.



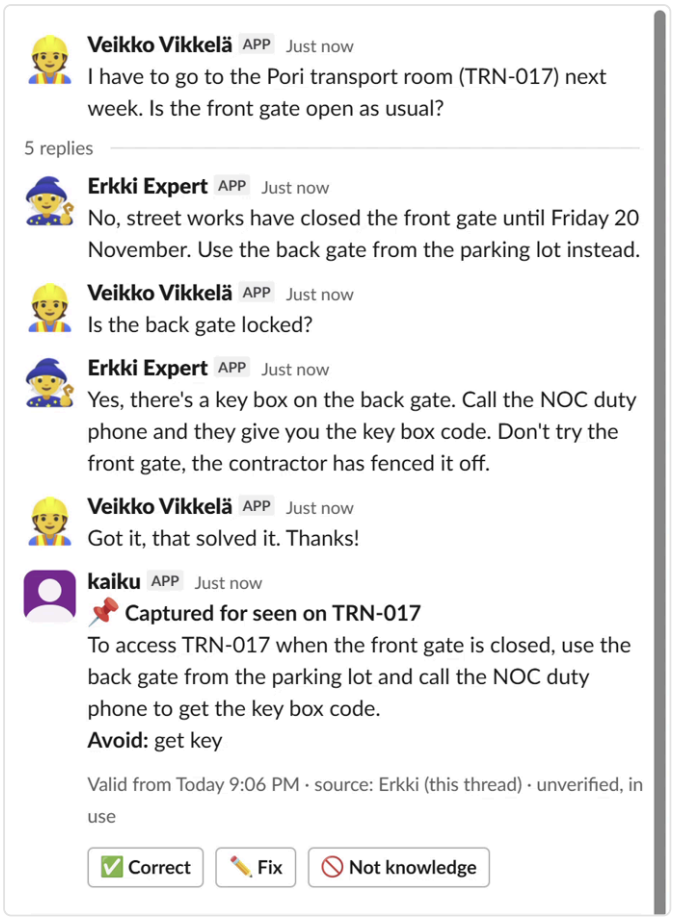
Kaiku architecture · 3 source types, 6 pipeline stages, 3 consumer types

Knowledge flows top to bottom: source threads are triaged and extracted, checked by gates V1 to V6, and written to Neo4j only if the temporal invariants hold. Telemetry is consulted twice, at capture (V4) and after an agent acts (outcome verification), so the network itself confirms or refutes captured knowledge.

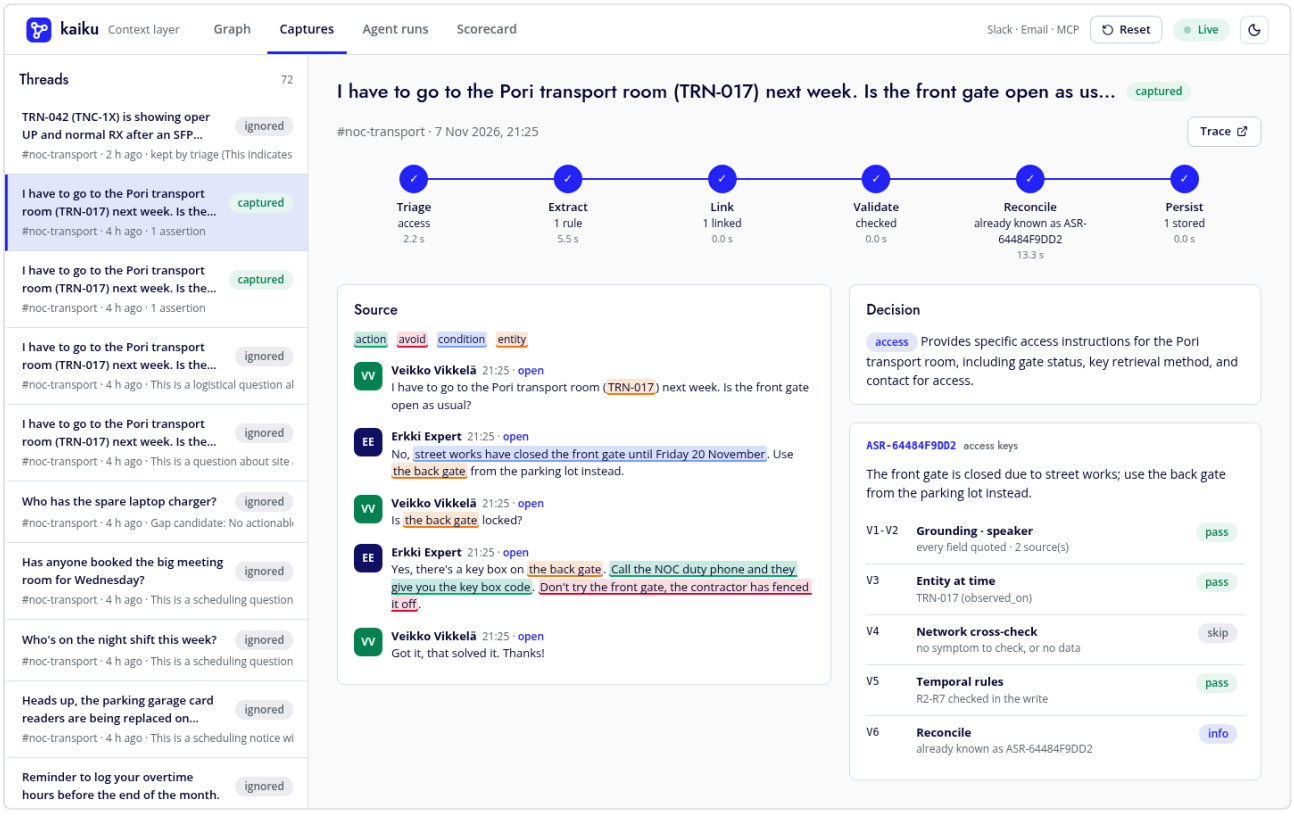
How it works in practice

An expert's answer in Slack becomes verified knowledge within about a minute, and an agent cites it the next time the same situation occurs. The screenshots below are from the running system on synthetic data.

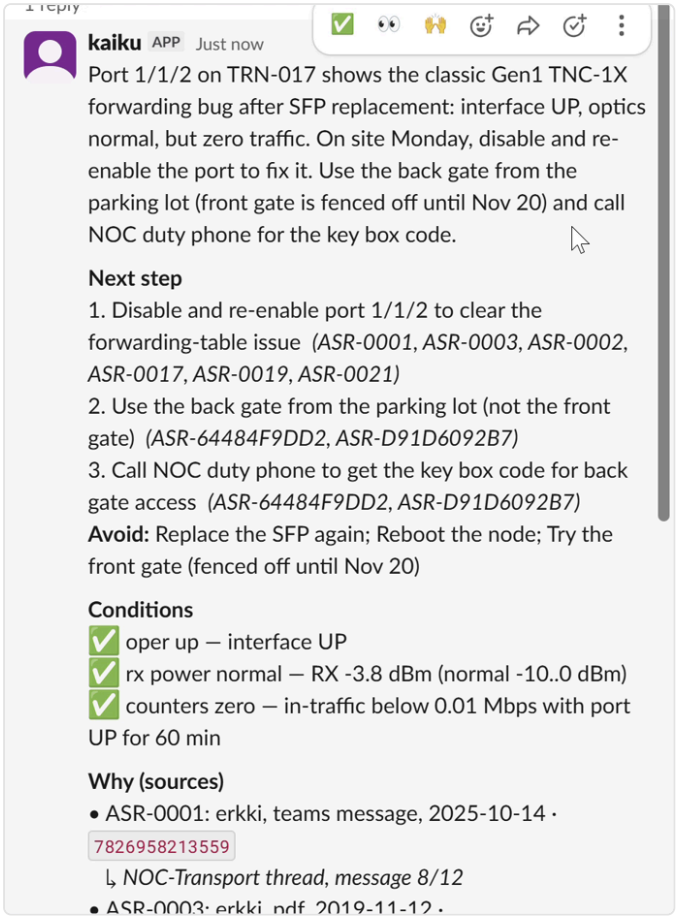
1. An expert answers in Slack and Kaiku captures it. Nobody fills in a form. Kaiku posts one card in the thread with the captured rule, what to avoid, its validity and its source; the buttons are optional.



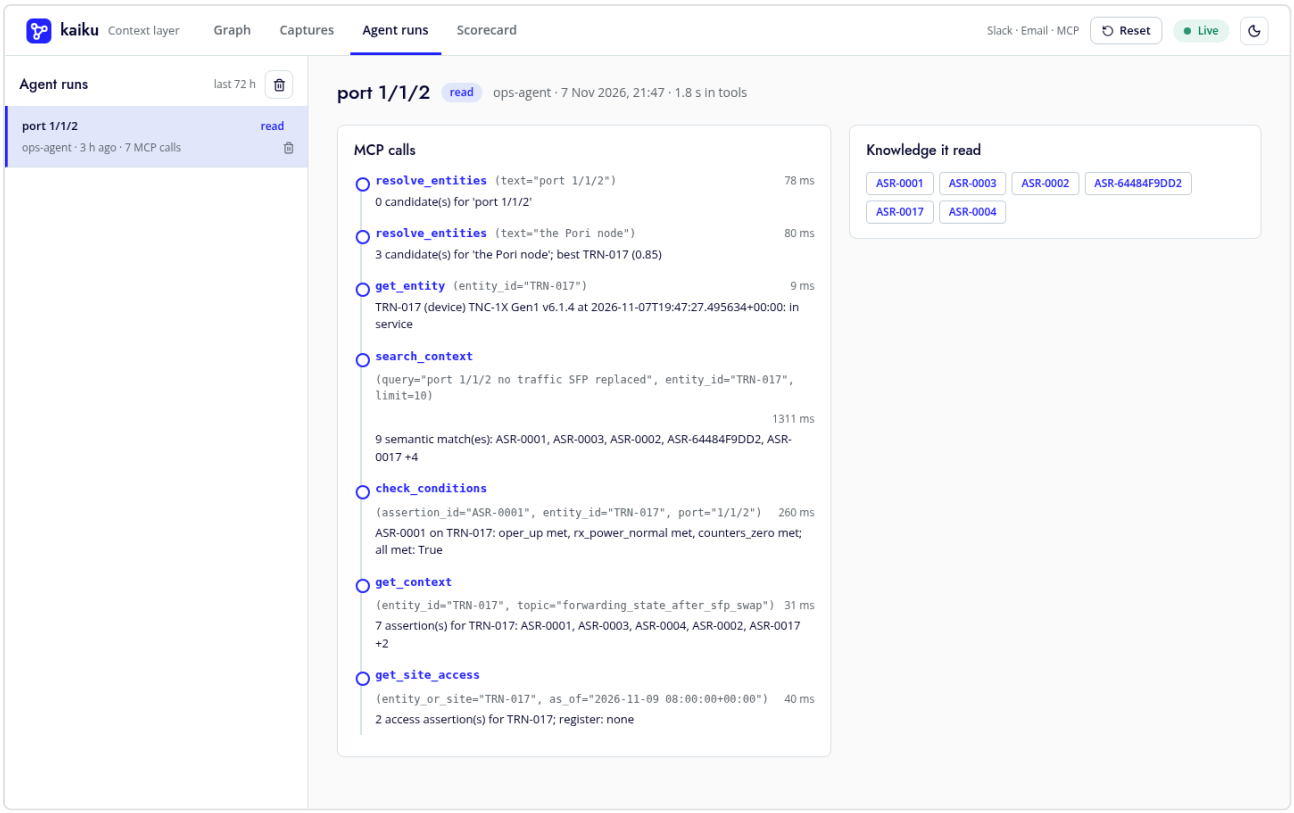
2. Every capture is traceable. The console shows each pipeline stage and gate result, with the source words that back each field highlighted.



3. An agent retrieves the knowledge and cites it. Asked about a fault on another node of the same hardware model, the ops agent gives the next step, what to avoid, the conditions checked against live telemetry, and the source messages.



4. Every agent call is auditable. The console lists each MCP call the agent made and the knowledge it read.



Capture and verification

Capture is passive: Kaiku reads Slack channels and an email drop folder live, and backfills Teams exports, Outlook archives, PDF notes, Confluence and ITSM KB articles with their original timestamps. A thread is processed when it is resolved ("Traffic is back", a ✅ reaction) or idle for 10 minutes.

Each thread runs through a LangGraph pipeline: **triage** (small model) → **extract** (chat model) → **link** → **validate** → **reconcile** → **persist**. The model proposes; deterministic gates decide what is stored:

Gate	What it guarantees
V1 Grounding	Every field quotes the source verbatim; ungrounded fields are dropped
V2 Speaker	A rule comes from an answer, not a question; a rejected suggestion becomes an *avoid* item
V3 Binding	Each linked device, site or segment exists at the start of validity and is unambiguous
V4 Telemetry	Reported symptoms are checked against the last complete 15-minute interval
V5 Time rules	23 temporal invariants run inside the write transaction; any violation rolls it back

Gate	What it guarantees
V6 Reconcile	The same knowledge from another channel corroborates instead of duplicating; a newer change supersedes; a disagreement is marked *disputed*

Dates are resolved by code, not the model: "until Friday 20 November" becomes an exclusive end at 00:00 Helsinki time. Nothing is deleted; corrections are retracted with a timestamp, so the history stays auditable.

The network confirms the knowledge. When an agent reports that it followed advice, Kaiku reads that port's telemetry for the next 30 minutes. Recovery counts as support; a failure counts against the rule only if its conditions were met. Confidence is the Beta posterior mean, shown with every answer:

$$\text{confidence} = \frac{1 + n_{\text{support}}}{2 + n_{\text{support}} + n_{\text{refute}}}$$

Retrieval

Retrieval filters by time and entity first and ranks second, so knowledge that was not valid at the requested moment can never be returned, and ranking never hides a valid fact.

Two time axes. Every assertion has a valid time (when it is true in the network; open-ended by default) and a record time (when Kaiku learned it). Queries take `as_of` and `known_at`, both defaulting to now. This answers both "what applies on the visit date?" and "what could the agent have known when it dispatched the engineer?"

Scope. For a device, Kaiku walks the graph at time t and weights each path:

Knowledge bound to	Weight
The device itself	1.00
Its hardware model or generation	0.90
Its site	0.85
A connecting fibre or microwave segment	0.80
Another device of the same model	0.50
Anything else valid (search only)	0.40

Rules whose model, generation or firmware qualifiers do not match what the device runs at t are dropped, so a Gen1-only rule never reaches a Gen3 node. Free-text queries are resolved to entities deterministically (ids, an alias table, fuzzy matching, place names).

Ranking. Similarity is exact cosine over every valid candidate (Qwen3-Embedding-8B, 1024 dimensions), multiplied by a structural score:

$$\text{score} = \max(0, \cos(q, a)) \times w_{\text{match}} \times w_{\text{status}} \times w_{\text{confidence}}$$

A disputed assertion (0.7) ranks below a believed one (1.0) even when its wording is closer. There is no age decay: a fact holds until it is superseded or expires. A superseded assertion is never shown without its successor. Without an embedding provider, keyword overlap replaces cosine and the result says so.

The console's graph view applies the same rules: drag the *As of* slider to see what was valid at any moment.



Agent interface

Agents use Kaiku only through MCP, Kaiku's own ops agent included, so any MCP-capable agent gets the same capability. Each agent has its own bearer token and every write is attributed to it.

Kind	Tools
Read	`get_context`, `search_context`, `get_site_access`, `get_entity`, `get_events`, `get_history`, `check_conditions`, `resolve_entities`
Write	`report_outcome`, `propose_assertion` (stored unverified), `open_knowledge_gap`, `flag_assertion`

Agents can add evidence and proposals but cannot confirm, retract or change validity. The reference agent's safety rules are enforced in code, not only in the prompt: no step without a citation to retrieved knowledge, no step whose checked conditions are not met, and no steps when the answer is "not documented".

Evaluation

Kaiku is evaluated at two levels: retrieval on its own, and the full path from raw conversation to agent answer. Scoring is exact match with no LLM judge, and end-to-end runs start from an empty graph, so the gold answers never leak.

Set	What it measures
Retrieval only (gold graph)	`search_context` ranking and time filtering, isolated from extraction
T0: Elisa's own example	Capture, linking, timing and agent recall on the challenge's reference case
T2: held-out capture (hand-written)	Extraction, linking and validity on threads never used for tuning
T2: held-out question answering	Agent answers through MCP, including questions with no documented answer

- **Capture:** time fields are extracted reliably; contradictions and the second assertion in multi-rule threads are often missed.
- **Agent answers:** access questions are answered correctly on both sides of a change date. One undocumented question received a confident answer, the most serious failure for autonomous operations.

End-to-end runs score each item once on a small synthetic held-out set, so their findings are indicative.

Technology stack

Layer	Choice
Graph and vectors	Neo4j 2026.10 with versioned Cypher migrations
Telemetry	DuckDB over 15-minute PM data and the simulator's live series
API and MCP	Python 3.12, FastAPI, MCP Python SDK (Streamable HTTP)
LLM orchestration	LangChain and LangGraph; provider set by configuration
Models (default, via Featherless)	Kimi-K2-Instruct for extraction and the agent, Qwen3-4B for triage, Qwen3-Embedding-8B
Chat surface	Slack Bolt in Socket Mode, no public URL
Console	React 19, TanStack Query, Cytoscape.js
Quality and ops	mypy strict, ruff, 394 backend tests on a real Neo4j, GitHub Actions CI, Phoenix tracing, Docker Compose

The run-book and configuration are in the repository README.

Limitations and next steps

Kaiku is a prototype on synthetic data. The time model and architecture are built for production; extraction accuracy and scale are not yet.

Limitation	Next step
Capture misses contradictions and multi-rule threads	Re-score the avoid-list fix; target contradiction and supersession detection
One confident answer to an undocumented question	Code-level check that every step is grounded in knowledge about the asked entity
Single runs, small synthetic held-out set	Three runs with variance; a larger set written by Elisa's experts
Exact cosine sized for about 100 assertions	Filtered vector index with the validity predicate inside the index
No live Teams capture (Slack stands in)	Microsoft Graph connector behind the existing ingest interface
Message text goes to a hosted LLM provider	An approved provider or self-hosted models for real operational data

All services bind to localhost, agents authenticate with per-agent tokens, and every write is an auditable event.